

Gestão de Estado

Android (Kotlin)

Instrutor: Ralf Lima



O que é Gestão de Estado?

É a prática de armazenar, modificar e expor os dados que determinam o que a interface do usuário (UI) exibe.

Isso abrange desde variáveis simples, como um estado de carregamento, até listas complexas de dados buscados em um servidor.

Principais comandos



Criação e Delegação de Estado

- mutableStateOf(valor): Cria um objeto observável pelo Compose. Quando o valor interno muda, o Compose redesenha a tela automaticamente.
- by: Permite ler e escrever na variável de estado diretamente, sem precisar digitar .value.
- remember{ }: Retém o valor na memória durante as recomposições. Sem ele, a variável reinicia com o valor padrão toda vez que a tela atualiza.
- rememberSaveable{ }: Funciona como o remember, mas salva o estado mesmo se o usuário girar a tela ou se o sistema destruir o app por falta de memória.



Coleções de Estado (Listas e Mapas)

- `mutableStateListOf<T>()`: Cria uma lista observável pelo Compose. Adicionar ou remover itens atualiza a tela instantaneamente.
- `mutableStateMapOf<K, V>()`: Cria um mapa (chave/valor) observável pelo Compose.



Integração com Fluxos Assíncronos (Reativos)

- `collectAsState()`: Transforma um `StateFlow` ou `Flow` do Kotlin Coroutines em um Estado que o Compose consegue ler.
- `collectAsStateWithLifecycle()`: Semelhante ao anterior, mas interrompe a coleta quando o app vai para o segundo plano. É a forma mais segura e recomendada para produção.
- `observeAsState()`: Transforma um `LiveData` (arquitetura mais antiga do Android) em um Estado do Compose.

Otimização de Leitura (Derivação)

- derivedStateOf{ }: Cria um estado que depende de outras variáveis de estado. Ele só avisa a UI se o resultado final da expressão mudar (ex: calcular se um botão deve ficar ativo com base no tamanho de uma lista).

Coroutines





O que são Coroutines

Coroutines (ou Corrotinas) são recursos do Kotlin para executar tarefas em segundo plano de forma assíncrona e não bloqueante.

Funcionam como "threads leves". Em vez de travar a tela do aplicativo enquanto espera uma resposta da internet ou do banco de dados, a coroutine pausa sua execução, libera o aplicativo para continuar respondendo ao usuário e retoma o trabalho assim que o resultado fica pronto.

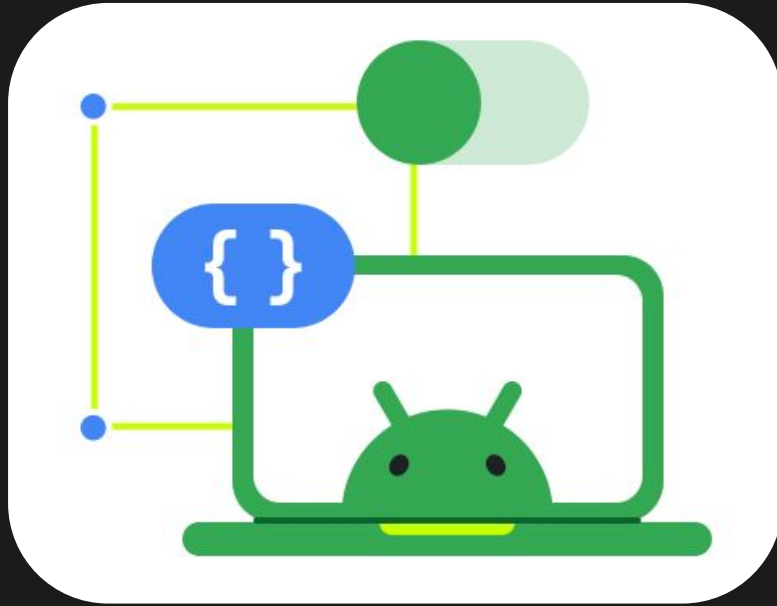
Características do Coroutines

- Leveza Extrema: Você pode rodar centenas de milhares de coroutines ao mesmo tempo sem estourar a memória. Elas compartilham a mesma Thread do sistema em vez de criar novas Threads pesadas.
- Código Sequencial: O código é escrito de cima para baixo usando funções normais, facilitando drasticamente a leitura e a manutenção.
- Funções Suspensas (suspend): Essas funções têm o poder de pausar a execução da coroutine sem travar a linha de processamento principal (Main Thread).

Características do Coroutines

- Cancelamento Cooperativo: Se o usuário sair de uma tela antes de uma foto carregar, a coroutine que estava baixando a imagem pode ser cancelada automaticamente, evitando desperdício de bateria e memória.
- Gerenciamento por Escopos (CoroutineScope): Toda coroutine precisa rodar dentro de um escopo que controla seu ciclo de vida. No Compose, usamos o LaunchedEffect ou rememberCoroutineScope.

UDF





O que é UDF

O UDF (Unidirectional Data Flow) ou Fluxo de Dados Unidirecional, é um padrão de design onde o estado da tela flui apenas em uma direção e as ações do usuário fluem na direção oposta.

Em vez de a interface do usuário alterar diretamente as variáveis, ela apenas avisa que algo acontece.

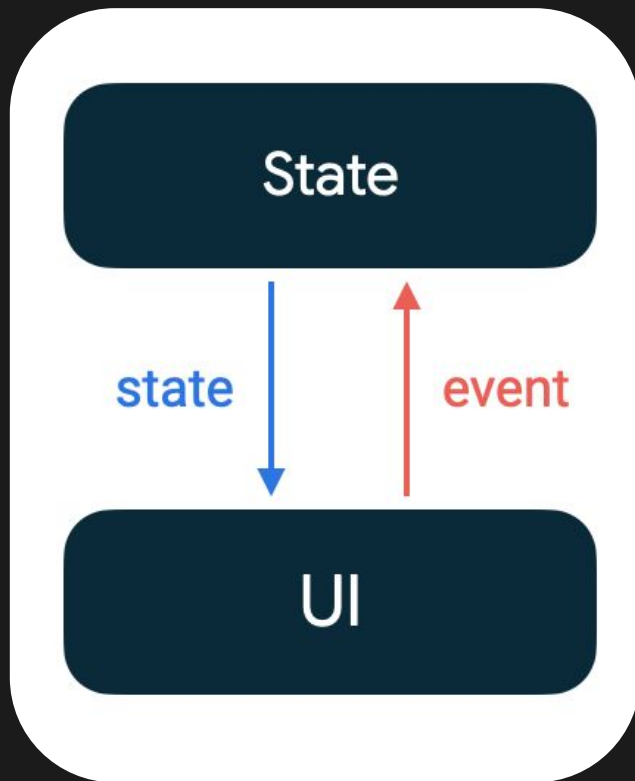


Funcionamento do UDF

- Estado Flui para Baixo (State Flows Down): O componente visual (UI) recebe os dados prontos de um local centralizado. A tela é "burra", ela apenas lê o estado atual e se desenha.
- Eventos Fluem para Cima (Events Flow Up): Quando o usuário interage com a tela (clica em um botão ou digita um texto), a UI não muda nada sozinha. Ela dispara um evento (geralmente uma função lambda) para avisar quem controla o estado.



Funcionamento do UDF

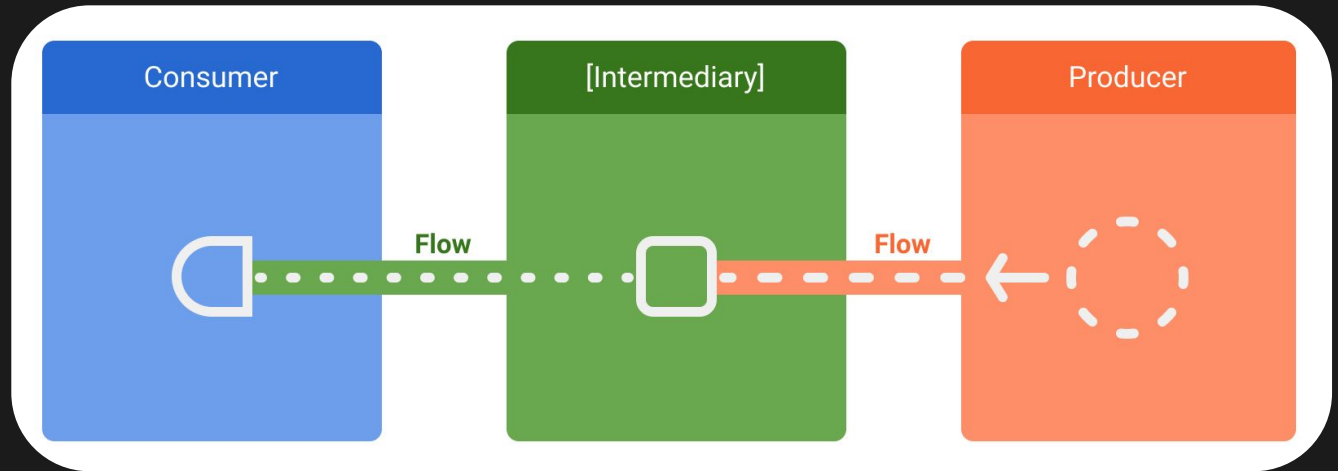




Motivos para utilizar o UDF

- Fonte Única da Verdade (Single Source of Truth): Como o estado fica centralizado em um só lugar, você evita bugs onde uma parte da tela exibe uma informação e outra parte exibe algo diferente.
- Desacoplamento: A interface não sabe de onde vêm os dados, ela só precisa do objeto de estado para funcionar.
- Casamento Perfeito com o Compose: O Jetpack Compose foi projetado do zero para funcionar com UDF. Reage automaticamente quando o estado que flui para baixo sofre qualquer alteração.

StateFlow





O que é StateFlow

O StateFlow é uma ferramenta do Kotlin Coroutines criada especificamente para implementar o UDF em projetos reais.

Se o UDF é o conceito de que o estado deve fluir para baixo, o StateFlow é o cano por onde esse estado corre da lógica de negócios até a tela.

No Android moderno, ele substituiu o antigo LiveData.



Características do StateFlow

- Sempre Tem um Valor Inicial: Você é obrigado a passar um estado padrão ao criá-lo, ele nunca nasce vazio (null).
- Retém o Último Estado: Se a tela for destruída e recriada (como ao girar o celular), o StateFlow mantém o último valor na memória. Assim que a tela reconecta, ela exibe o dado instantaneamente, sem precisar recarregar da internet.
- Evita Redesenhar à Toa (Distinct Until Changed): Se você enviar o exato mesmo dado para ele, ele ignora. Ele só avisa o Compose se o valor realmente mudar, economizando processamento do celular.

Vamos praticar!

